

ATTORNEY.AI

Research Reference Guide

[All Repos](#) · [Datasets](#) · [Papers](#) · [Free Stack](#) · [Build Order](#)

Muhammad Usama | SP23-BCS-069

COMSATS University Islamabad — Department of Computer Science

Supervisor: Dr. Tehseen Riaz Abbasi

Compiled: 25 April 2026

How to use this guide

This document summarises every repository, dataset, and paper researched for the ATTORNEY.AI Final Year Project. Each entry tells you exactly what the resource is and — more importantly — what you should use it for. Repos are grouped into tiers so you know the order to tackle them.

Symbol / Color	Meaning
Tier 1 (dark green)	Clone and build your pipeline from these first
Tier 2 (teal)	Borrow one specific feature from each
Tier 3 (amber)	Tools for ingesting Pakistani legal PDFs into Chroma
Tier 4 (blue)	Read the paper/docs only — do not clone
Skip (red)	Wrong stack or superseded — do not waste time
★ Your own assets	These are unique to your project — protect them

At a glance

20+ Repos evaluated	2 Datasets	1 Academic paper	3 Tier 1 repos (build from)	100% Free stack
------------------------	---------------	---------------------	-----------------------------------	--------------------



Your Own Assets

These two resources are what make ATTORNEY.AI an original academic contribution. Every other project in this guide helps you build the pipeline. These are what make your pipeline Pakistani and legally accurate.



muhammadusamahoyrr / pakistan-legal-dataset

github.com/muhammadusamahoyrr/pakistan-legal-dataset

Your dataset ★

Pakistan-specific ✓

Province metadata ✓

Real court data ✓

What it is:

Real Pakistani court judgments collected by YOU — organised into folders by court: Balochistan High Court, Islamabad High Court, Lahore High Court, Peshawar High Court, Sindh High Court, and Supreme Court Decisions. Uploaded recently. First Pakistani multi-court legal dataset on GitHub.

Use for:

This is your primary Chroma knowledge base. Ingest all PDFs using `llmsherpa` and `Unstructured`. The folder names give you free metadata — court, province, and case type. This enables province-level retrieval: when a user says 'Lahore', your system filters Chroma to only Lahore High Court and Supreme Court documents. No other repo has this.



nlp-anonymous-researcher / LEGAL-UQA (HuggingFace Dataset)

huggingface.co/datasets/nlp-anonymous-researcher/LEGAL-UQA

Your asset ★

Pakistan law ✓

619 Q&A; pairs ✓

Bilingual EN+UR ✓

Free download ✓

What it is:

619 parallel English-Urdu question-answer pairs from Pakistan's constitution. Created using OCR, manual review, and GPT-4 translation. Testing showed Claude-3.5-Sonnet achieved 99.19% accuracy on this dataset. The best embedding model for Pakistani legal text is OpenAI text-embedding-3-large.

Use for:

Two uses: (1) Seed your Chroma knowledge base — extract the `context_eng` and `context_urdu` columns and load them as pre-chunked constitutional law documents. (2) Evaluate your RAG — feed all 619 questions into your pipeline and compare the answers against the ground truth. Report this accuracy score to your supervisor. Also cite in your SRS to justify choosing Gemini or Claude as your LLM.

1

Tier 1 — Core Pipeline

Start with Repo 1 (GiovanniPasq) as your skeleton. Add Repo 3 (sougaat) as your retrieval engine. Use Repo 2 (wassim249) as your FastAPI backend wrapper. Together these three give you a working conversational legal RAG system.

1

GiovanniPasq / agentic-rag-for-dummies

github.com/GiovanniPasq/agentic-rag-for-dummies

Follow-up
questions ✓

Clarification node
✓

LangGraph ✓

Multi-hop

HITL

New release!

What it is:

A modular Agentic RAG built with LangGraph. Published just days ago. It has an explicit 'request_clarification' node — the AI pauses mid-conversation and asks the user for more information before doing any retrieval. The full flow is: User Query → Summary → Rewrite Query → Ask Clarification → Parallel Retrieval → Final Answer. Every part is swappable via a config file.

Use for:

This is your skeleton. Copy the `rag_agent/nodes.py` file — specifically the `request_clarification` node and the `rewrite_query` node. Change the clarification prompt to ask Pakistani legal intake questions such as: Is this a civil or criminal matter? Which province are you in? Has an FIR been filed? Replace the Gradio UI wiring with a FastAPI WebSocket that sends the question to your Next.js frontend. Swap the LLM to Gemini Flash or Ollama. Start here first.

2

wassim249 / fastapi-langgraph-agent-production-ready-template

github.com/wassim249/fastapi-langgraph-agent-production-ready-template

FastAPI + LangGraph ✓

JWT auth ✓

Rate limiting ✓

Langfuse tracing

Docker ready

What it is:

A production-ready FastAPI + LangGraph backend template published days ago. Includes JWT authentication, per-user rate limiting (via `slowapi`), long-term memory (`mem0`), LLM call tracing (Langfuse), exponential backoff on errors, and Docker deployment. Folder structure: `api/v1/` routes, `core/langgraph/` graph, `prompts/` folder.

Use for:

This is your FastAPI backend structure. The JWT auth and session management are already done. The rate limiting closes the LLM API overuse gap in your SRS. The Langfuse tracing lets you answer the supervisor question: How do you monitor AI responses? Replace the generic agent inside this template with your legal RAG agent from GiovanniPasq above.

3

sougaaat / RAG-based-Legal-Assistantgithub.com/sougaaat/RAG-based-Legal-Assistant

Legal domain ✓

Hybrid BM25+dense ✓

Query classifier ✓

RRF reranking

RAGAS eval

What it is:

A legal-domain RAG system with hybrid BM25 + semantic retrieval, multi-hop and multi-query strategies, Reciprocal Rank Fusion (RRF) for reranking, a query complexity classifier, and a RAGAS evaluation setup. Legal domain specific.

Use for:

This is your retrieval engine. Hybrid BM25 + dense search is essential for Pakistani legal text because statute references like 'Section 302 of PPC' are keyword matches, not semantic ones. Pure vector search will miss them. Plug this repo's retrieval modules into GiovanniPasq's retrieval step and replace its vector store with your Chroma instance. Use `decide_query_complexity.py` as the router that decides: simple query = single retrieval, complex query = multi-hop. Use the RAGAS setup to evaluate against your 619 LEGAL-UQA Q&A pairs.

2

Tier 2 — Specific Pieces

4

nilsjennissen / langgraph

github.com/nilsjennissen/langgraph

LLM + LangGraph wiring ✓

EnsembleRetriever

Configurable weights

What it is:

The only repo across all our research that has ANTHROPIC_API_KEY in its .env file — it directly connects Claude (or any LLM) into a LangGraph state graph. Also has an EnsembleRetriever that combines keyword and semantic search with configurable weights per use case.

Use for:

Copy the LLM + LangGraph wiring pattern. Replace Claude with Gemini Flash for free use. Set the EnsembleRetriever keyword weight to 0.6 for Pakistani legal text because statute references need keyword matching, not just semantic similarity.

5

junfanz1 / Cognito-LangGraph-RAG-Chatbot

github.com/junfanz1/Cognito-LangGraph-RAG-Chatbot

Hallucination control ✓

Relevance grader ✓

CRAG pattern

Answer grader

What it is:

A Corrective RAG (CRAG) system with LangGraph. It grades every retrieved document for relevance before answering, rewrites the query if the docs are not relevant, and checks whether the final answer is actually grounded in the retrieved text. This prevents the AI from making up legal information.

Use for:

Copy the GRADE_DOCUMENTS node and the hallucination grader prompt. Wire it between your retrieval step and the LLM generation step. Rule: if relevance grade is below 0.75, route back to the clarification node. If the answer grader fails, refuse to answer and suggest the user consult a real lawyer. This is your legal safety net.

6

JoshuaC215 / agent-service-toolkitgithub.com/JoshuaC215/agent-service-toolkit

LangGraph interrupt() ✓

Chroma RAG agent ✓

FastAPI streaming

Next.js fork

What it is:

A full FastAPI + LangGraph toolkit that uses the newest LangGraph v1.0 interrupt() API for human-in-the-loop pausing. Has a pre-built ChromaDB RAG agent, streaming endpoints (token by token), and a community fork with a Next.js frontend.

Use for:

Study the interrupt() pattern — this is the correct LangGraph v1.0 way to pause the graph and wait for the user's answer before continuing. The ChromaDB RAG agent file (src/agents/rag_agent.py) is your starting point for connecting Chroma to your pipeline. Copy the streaming endpoint design for your Next.js WebSocket integration.

7

sardanaaman / langgraph-compassgithub.com/sardanaaman/langgraph-compass

Follow-up questions ✓

Plug-in module ✓

LangGraph compatible

What it is:

A standalone module that generates follow-up questions for conversational LangGraph agents. Designed specifically as a plug-in component, not a full application. Found via the awesome-LangGraph directory.

Use for:

Drop this directly into your LangGraph pipeline as the clarification node. It saves you writing the follow-up question generation logic from scratch. Customise the prompt to ask Pakistani legal intake questions.

8

sfantaye / legalmindgithub.com/sfantaye/legalmind

Legal + LangGraph ✓

Document generation ✓

FastAPI backend

What it is:

A legal assistant that combines FastAPI, LangGraph stateful workflows, and document generation for NDAs and rental agreements. The closest existing open-source project to ATTORNEY.AI's full scope.

Use for:

Study the document generation pipeline. It shows how to take structured user inputs and produce a complete legal document via LLM. Apply the same pattern to your docxtempl + LibreOffice document automation module (ATTORNEY.AI Module 6).

9

arulkumarann / legalRAGgithub.com/arulkumarann/legalRAG

Session isolation ✓

/sources endpoint ✓

Legal domain

FastAPI clean structure

What it is:

A FastAPI legal RAG with session-based handling where every user session has its own isolated vector search scope. Has a clean `/sources/{session_id}` endpoint that returns which documents were retrieved for any given chat.

Use for:

Copy two things only. First: the session-per-user isolation pattern — this prevents one client's case data from leaking into another client's chat. Second: the `/sources` endpoint design — this is your citation return API that shows users which court judgments were used to answer their question. Swap Pinecone with Chroma.

10

sergio11 / langgraph_legal_assistantgithub.com/sergio11/langgraph_legal_assistant

Jurisdiction routing ✓

Multi-agent ✓

MIT license

Legal domain

What it is:

A multi-agent LangGraph legal system that routes queries by jurisdiction, then runs specialist agents to find applicable laws, case precedents, and relevant document templates. MIT license.

Use for:

Copy the jurisdiction-based routing logic. In your system, replace generic jurisdiction with Pakistani province + court level (Punjab/Sindh/KPK/Federal). The multi-agent pattern — one agent for statutes, one for precedents, one for templates — maps directly to your Lawyer AI Assistant module (M19 in your SRS).

3

Tier 3 — Document Ingestion Tools

Pipeline order: Your PDF → UTRNet (only if scanned Urdu) → Unstructured (parse) → llmsherpa (structure-aware chunking) → Embed with multilingual-MiniLM → Store in Chroma with court/province metadata.

A

nlmatics / llmsherpa

github.com/nlmatics/llmsherpa

Structure-aware chunking ✓

Legal PDF ✓

Hierarchy preserved ✓

What it is:

A smart PDF parser that preserves document structure during chunking. It keeps sections, headings, and sub-clauses together instead of splitting them arbitrarily at character limits. Critical for legal PDFs.

Use for:

Use as your primary chunking tool for all court judgment PDFs. It keeps legal section hierarchy intact — so 'Section 302 subsection (a)' stays as one coherent chunk and is not split mid-sentence. Run this AFTER Unstructured has pre-processed the PDF.

B

Unstructured-IO / unstructured

github.com/Unstructured-IO/unstructured

Multi-format ✓

Urdu support ✓

LEGAL-UQA paper validated ✓

15+ formats

What it is:

A universal document parser supporting 15+ file formats including PDF, DOCX, HTML, and EPUB. Handles mixed English-Urdu documents and varied PDF formatting from different Pakistani courts. Referenced in the LEGAL-UQA paper.

Use for:

Run this first on all your court judgment PDFs before llmsherpa. It normalises the raw PDF output so that llmsherpa gets clean, consistent input. Particularly important for Islamabad High Court PDFs which have complex formatting.



abdur75648 / UTRNet-High-Resolution-Urdu-Text-Recognition

github.com/abdur75648/UTRNet-High-Resolution-Urdu-Text-Recognition

Urdu OCR ✓

LEGAL-UQA paper validated ✓

Only for scanned PDFs

What it is:

A high-resolution Urdu OCR model. Referenced in the LEGAL-UQA paper. Only relevant when your court PDFs are scanned images rather than digital text — which is common for older Pakistani court records.

Use for:

Test your PDFs first by running `pdftotext` on one. If it returns readable text, skip UTRNet entirely. If the output is garbled or empty, your PDF is a scanned image — run UTRNet first to extract the Urdu text, then pass the result through Unstructured and Llmsherpa.

4

Tier 4 — Reference Only

R1

hoorangyee / LRAGE

github.com/hoorangyee/LRAGE

ACL paper

Legal eval methodology

Read only

What it is:

A research-grade RAG evaluation framework for the legal domain. Submitted to ACL. Covers BM25 and dense retrieval, reranking, and LLM-as-judge evaluation across legal datasets like LegalBench and Pile-of-Law.

Use for:

Read the paper only — not the code. When your supervisor asks 'How do you know your RAG is accurate?', cite this evaluation methodology. It gives your pipeline academic credibility without any extra implementation work.

R2

LexStack-AI / LexReviewer

Found via: github.com/von-development/awesome-LangGraph

Citation highlighting

Legal + LangGraph

Demo-worthy pattern

What it is:

A LangGraph-powered legal PDF RAG service with hybrid retrieval and citation-aware answers. Its standout feature is bounding-box citation highlighting — it highlights the exact sentence in the source document that answered the question.

Use for:

Study the citation bounding-box design for your panel demo. Showing a user exactly which line in which court judgment answers their question is far more impressive than just listing document names. Even a simplified version of this in your demo will stand out.

R3

von-development / awesome-LangGraph

github.com/von-development/awesome-LangGraph

Living directory

Updated weekly

1700+ stars

What it is:

A curated directory of the entire LangGraph ecosystem — 1,700+ stars, 187 forks, updated every week. Lists community projects by category: RAG, legal assistants, FastAPI templates, HITL, document processors.

Use for:

Bookmark this page. Whenever you hit a specific implementation problem during development, search the README for the relevant keyword — legal, streaming, HITL, FastAPI, Chroma — and you will find a purpose-built community repo that solves it.

Repos to skip entirely

These were evaluated and ruled out. Do not spend time on them.

Repository	Why skip
MohammedAly22 / AgenticRAG	Uses Cohere + Streamlit. No clarification node. Wrong stack.
Akash-47-tank / LegalRAG	Flask backend, zero stars, generic Q&A with no legal logic.
Muhammad2003 / Llama3-LegalLM	Fine-tuning approach — completely different from RAG. Not relevant.
moeedfaiz / Multi-PDF-RAG-Agent	No FastAPI, no legal domain, no LangGraph. Too generic.
sky787770 / Agents	Good concepts but superseded by GiovanniPasq (newer + better).
ygouvergmail / LIA-Assistant	AGPL-3.0 license. 90% of codebase is irrelevant to your scope.
mt5-large-legal-uqa (HuggingFace)	4.92 GB model. Unnecessary — Gemini/Llama beats it for free.

This is the most important academic reference for your FYP. It validates your LLM choice, gives you a ready-made evaluation dataset, and provides the exact tools used to process Pakistani legal text. Cite it in your SRS.

Paper details

Title	LEGAL-UQA: A Low-Resource Urdu-English Dataset for Legal Question Answering
Authors	Faizan Faisal, Umair Yousaf
Published	October 2024 arXiv:2410.13013
arXiv URL	arxiv.org/abs/2410.13013
Dataset URL	huggingface.co/datasets/nlp-anonymous-researcher/LEGAL-UQA
Key finding	Claude-3.5-Sonnet achieved 99.19% human-evaluated accuracy on Pakistani constitutional legal Q&A;
Best embedding	OpenAI text-embedding-3-large outperforms other models on Pakistani legal text
Dataset size	619 Q&A; pairs, parallel English + Urdu, sourced from Pakistan's constitution

Tools used in this paper (cite these in your SRS methodology)

Footnote	Tool	GitHub URL	Your use
8	UTRNet — Urdu OCR	github.com/abdur75648/UTRNet-High-Resolution-Urdu-Text-Recognition	OCR text recognition of PDFs
9	Unstructured — PDF parser	github.com/Unstructured-IO/unstructured	Pre-process all court PDFs
10	llmsherpa — PDF chunker	github.com/nlpmatics/llmsherpa	Structure-aware chunking
12	mt5-base-finetuned-urdu	huggingface.co/eslamxm/mt5-base-finetuned-urdu	Baseline only — do not use in your pipeline

How to cite in your SRS

Faisal, F., & Yousaf, U. (2024). LEGAL-UQA: A Low-Resource Urdu-English Dataset for Legal Question Answering. arXiv preprint arXiv:2410.13013.

Decision: Do not use Claude API or OpenAI (paid). Use Gemini Flash for demo day and Ollama locally during development. The LangChain abstraction means swapping LLMs is literally one line of code — you can switch at any time.

Free LLM options (ranked best to worst for your use case)

LLM	Where to get it	Free tier	Best for
Gemini 2.0 Flash	aistudio.google.com	15 req/min, 1M tokens/day	Panel demo day — best quality, still free
Ollama + Llama 3.1	ollama.com	Fully local, unlimited	Daily development — no internet, no limits
Groq + Llama 3.1 70B	console.groq.com	Generous free tier	Fast inference backup
HuggingFace API	huggingface.co	1000 req/day	Smaller models, last resort

Complete free stack — all components

Component	Free Tool	How to install / get it
LLM (demo)	Gemini 2.0 Flash	API key from aistudio.google.com — free, no card
LLM (dev)	Ollama + Llama 3.1 8B	ollama.com then: <code>ollama pull llama3.1</code>
Embeddings	multilingual-MiniLM-L12-v2	<code>pip install sentence-transformers</code> (runs locally)
Vector DB	ChromaDB	<code>pip install chromadb</code> (local, persistent)
Pipeline	LangGraph + LangChain	<code>pip install langgraph langchain</code>
Backend	FastAPI + Uvicorn	<code>pip install fastapi uvicorn</code>
Frontend	Next.js 15	<code>npx create-next-app</code> (open source)
GPU compute	Kaggle Notebooks	kaggle.com — 30 hours/week free GPU
PDF parsing	llmsherpa + unstructured	<code>pip install llmsherpa unstructured</code>
Monitoring	Langfuse (free tier)	langfuse.com — 50,000 observations/month free
Knowledge base	Your dataset + LEGAL-UQA	Your GitHub repo + HuggingFace download

LLM swap code — one line change between any free provider

```
# Option 1 – Gemini Flash (best for demo day) from langchain_google_genai import
ChatGoogleGenerativeAI llm = ChatGoogleGenerativeAI(model='gemini-2.0-flash',
google_api_key=GEMINI_KEY) # Option 2 – Ollama local (best for development, no internet needed)
from langchain_ollama import ChatOllama llm = ChatOllama(model='llama3.1') # first run: ollama
pull llama3.1 # Option 3 – Groq (fast inference backup) from langchain_groq import ChatGroq llm
= ChatGroq(model='llama-3.1-70b-versatile', groq_api_key=GROQ_KEY) # Embeddings – same for all
LLM choices (runs locally, handles English + Urdu) from langchain_community.embeddings import
HuggingFaceEmbeddings embeddings = HuggingFaceEmbeddings(
model_name='sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2' )
```

7

Your Build Order

Step 1

Get GiovanniPasq working with a free LLM

Clone GiovanniPasq/agentic-rag-for-dummies. Wire Ollama (local) or Gemini Flash (free API) using the nilsjennissen pattern. Get the clarification node triggering end-to-end with a simple test document. Do not move to Step 2 until this works.

Repos used: [GiovanniPasq](#) + [nilsjennissen](#) + [Ollama](#) or [Gemini Flash](#)

Step 2

Build your Chroma knowledge base on Kaggle

Use Kaggle's free GPU. Ingest the LEGAL-UQA 619 Q&A; pairs plus all your pakistan-legal-dataset PDFs using Unstructured and llmsherpa. Attach court/province metadata to every chunk. Export and download the Chroma DB folder.

Repos used: [LEGAL-UQA dataset](#) + [your pakistan-legal-dataset](#) + [llmsherpa](#) + [Unstructured](#)

Step 3

Plug in sougaaat's hybrid retrieval

Replace GiovanniPasq's retriever with the BM25 + Chroma hybrid from sougaaat. Add the decide_query_complexity.py router. Test retrieval against your legal knowledge base and confirm that statute references like 'Section 302 PPC' are returned correctly.

Repos used: [sougaaat](#) / [RAG-based-Legal-Assistant](#)

Step 4

Add the relevance grader and hallucination check

Copy the GRADE_DOCUMENTS node from junfanz1. Wire it between your retrieval step and the LLM generation step. Set the rule: grade below 0.75 routes back to clarification. If the answer grader fails, the system refuses and suggests a lawyer.

Repos used: [junfanz1](#) / [Cognito-LangGraph-RAG-Chatbot](#)

Step 5

Wrap everything in the production FastAPI backend

Use the wassim249 template as your FastAPI structure. Keep its JWT auth, rate limiting, session management, and Langfuse tracing. Expose your LangGraph agent through FastAPI WebSocket endpoints for the Next.js frontend.

Repos used: [wassim249](#) / [fastapi-langgraph-agent-production-ready-template](#)

Step 6

Evaluate against the LEGAL-UQA 619 Q&A; pairs

Feed all 619 questions from the LEGAL-UQA dataset into your completed pipeline. Compare the generated answers against the ground truth answers. Calculate and record the accuracy score. This number is your academic contribution — show it to your supervisor.

Repos used: [LEGAL-UQA dataset](#) + [LRAGE evaluation methodology](#)

8

Complete Pipeline Flow

1

User sends a query

Example: 'I have a property dispute with my landlord in Karachi'

v

2

LangGraph checks context completeness

Does the system have enough information to retrieve relevant documents accurately?

v

3

Clarification node fires (if needed)

AI asks: 'Is this a rental dispute or ownership dispute? Has any legal notice been served?'

v

4

Query is rewritten with full context

The richer, more specific query is sent to the hybrid retriever

v

5

Hybrid retrieval runs

Province filter applied (Sindh) → BM25 keyword match + Chroma semantic search + RRF reranking

v

6

Relevance grader scores the retrieved chunks

If score is below 0.75, loop back to Step 3 and ask another clarifying question

v

7

LLM generates answer with citations

Gemini Flash or Ollama generates the answer using only the retrieved court judgments

v

8

Hallucination check runs

Is the answer actually grounded in the retrieved documents? If not, refuse and suggest a lawyer

v

9

Response streamed to Next.js frontend

Answer delivered via FastAPI WebSocket with source citations (court name + case number)